

Correction — Réponses uniquement

1. GADT — Indexation

Exercice 1

```
let rec nth xs i =
  match xs, i with
  | Nil, _ -> failwith "index"
  | Cons (x, _), 0 -> x
  | Cons (_, xs'), i -> nth xs' (i - 1)
```

Exercice 2

```
let rec insert a i xs =
  match xs, i with
  | Nil, 0 -> Cons (a, Nil)
  | Nil, _ -> failwith "index"
  | Cons (x, xs'), 0 -> Cons (a, Cons (x, xs'))
  | Cons (x, xs'), i -> Cons (x, insert a (i - 1) xs')
```

Exercice 3

```
val nth2 : ('a,'n) nlist -> 'n below -> 'a
```

Exercice 4

```
let rec nth2 xs i =
  match xs, i with
  | Cons (x, _), Zero -> x
  | Cons (_, xs'), Succ i' -> nth2 xs' i'
```

Exercice 5

```
val insert2 :
  'a ->
  'n below ->
  ('a,'n) nlist ->
  ('a,'n succ) nlist
```

Exercice 6

```
let rec insert2 a i xs =
  match xs, i with
  | xs, Zero -> Cons (a, xs)
  | Cons (x, xs'), Succ i' ->
    Cons (x, insert2 a i' xs')
```

2. Monade non-déterministe

Exercice 7

```
let time_to_cross ms =
  List.fold_left
    (fun acc m -> max acc (Blackpink.speed m))
    0 ms
```

Exercice 8

```
let remove_list xs ys =
  List.filter (fun y -> not (List.mem y xs)) ys
```

Exercice 9

```
let select_people rive =
  let singles = List.map (fun x -> [x]) rive in
  let pairs =
    List.concat
      (List.map
        (fun x ->
          List.map
            (fun y -> [x; y])
            (List.filter
              (fun y -> Blackpink.compare x y < 0)
              rive))
        rive)
  in
  singles ++ pairs
```

Exercice 10

```
let neighbors (from_, to_, is_far, time_left) =
  select_people from_ >>= fun grp ->
  let t = time_to_cross grp in
  if t > time_left then zero
  else
    let from' = remove_list grp from_ in
    let to' = grp @ to_ in
    return (to', from', not is_far, time_left - t)
```

Exercice 11

```
let ok (_, _, is_far, _) =
  not is_far
```